Nano 33 BLE Sense Rev2 Cheat Sheet

Learn how to set up the Nano 33 BLE Sense Rev2, get a quick overview of the components, information regarding pins and how to use different Serial (SPI, I2C, UART) and Wireless (Wi-Fi®, Bluetooth®) protocols.

Author · Benjamin Dannegård Last revision · 11/15/2024

Board Package

Datasheet

Installation

Arduino IDE 1.8.X

Arduino IDE 2

Cloud Editor

MicroPython

Using OpenMV IDE

Forcing Bootloader

Pins

Analog Pins

PWM Pins

Digital Pins

5V Pin

IMU

BMI270 and BMM150

BMI270 and BMM150 Library

Accelerometer

Gyroscope

Magnetometer

Tutorials

Proximity and Gesture Detection

APDS9960

APDS9960 Library

Tutorials

Temperature and Humidity Sensor

HS3003

HS3003 Library

Tutorial

Pressure Sensor

LPS22HB

LPS22HB Library

Tutorial

Microphone

MP34DT06JTR

Help

RGB
Communication –
SPI

The Arduino® Nano 33 BLE Sense Rev2

This article is a collection of guides, API calls, libraries and tutorials that can help you get started with the Nano 33 BLE Sense Rev2 board.

You can also visit the **documentation platform for the Nano 33 BLE Sense Rev2**.

Board Package

The Nano 33 BLE Sense Rev2 uses the **Arduino Mbed OS Nano Board Package**.

Datasheet

The full datasheet is available as a downloadable PDF from the link below:

Download the Arduino Nano 33 BLE Sense Rev2 datasheet

Installation

Arduino IDE 1.8.X

The Nano 33 BLE Sense Rev2 can be programmed through the **Classic Arduino IDE 1.8.X**. To install your board, you can check out the guide below:

Arduino IDE 2

The Nano 33 BLE Sense Rev2 can be programmed through the **Arduino IDE 2**. To install your board, you can check out the guide below:

> How to use the board manager with the Arduino IDE 2

Cloud Editor

The Nano 33 BLE Sense Rev2 can be programmed through the **Cloud Editor**. To get started with your board, you will only need to install a plugin, which is explained in the guide below:

> Getting started with the Cloud Editor

MicroPython

This board is supported by MicroPython. Visit the MicroPython documentation for getting started with installation and usage.

Using OpenMV IDE

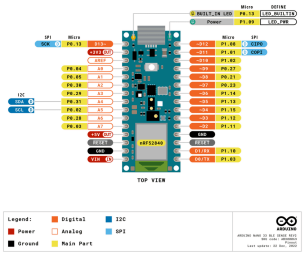
If you want to use your board with MicroPython and OpenMV. Follow the tutorial below.

> Getting started with OpenMV with Nano 33 BLE Sense Rev2

Forcing Bootloader

If this happens, we can double-tap the reset button, to forcefully trigger the bootloader.

Pins



The pinout for Nano 33 BLE Sense Rev2.

Analog Pins

The Nano 33 BLE Sense Rev2 has 8 analog pins, that can be used through the `analogRead()` function.

```
value = analogRead(pin, v
```

Please note: pin `A4` and `A5` should be used for I2C only.

PWM Pins

Pins **D2-D12** and **A0-A7** supports PWM (Pulse Width Modulation).

Pins A4, A5 and D11, D12 are not recommended for PWM as they have I2C & SPI buses attached.

```
analogWrite(pin, value);
```

Digital Pins

There are a total of 14 digital pins.

To use them, we first need to define them inside the `void setup()` function of our sketch.

```
pinMode(pin, INPUT); //co
pinMode(pin, OUTPUT); //c
pinMode(pin, INPUT_PULLUP
```

To read the state of a digital pin:

```
state = digitalRead(pin);
```

To write a state to a digital pin:

```
digitalWrite(pin, HIGH);
```

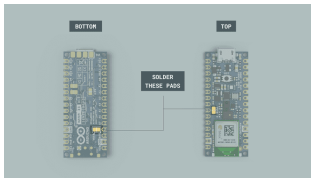
5V Pin

The microcontroller on the Arduino Nano 33 BLE Sense Rev2 runs at 3.3V, which means that you must never apply more than 3.3V to its Digital and Analog pins. Care must be taken when connecting sensors and actuators to assure that this limit of 3.3V is never exceeded. Connecting higher voltage signals

other Arduino boards, will damage the Arduino Nano 33 BLE Sense Rev2.

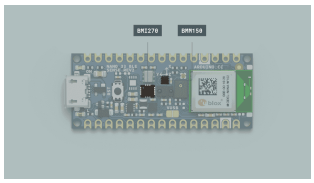
To avoid such risk with existing projects, where you should be able to pull out a Nano and replace it with the new Nano 33 BLE Sense, we have the 5V pin on the header, positioned between RST and A7 that is not connected as default factory setting. This means that if you have a design that takes 5V from that pin, it won't work immediately, as a precaution we put in place to draw your attention to the 3.3V compliance on digital and analog inputs.

5V on that pin is available only when two conditions are met: you make a solder bridge on the two pads marked as VUSB and you power the Nano 33 BLE Sense Rev2 through the USB port. There are two sets of VUSB pads on the Arduino Nano 33 BLE Sense Rev2, one set on the bottom and one set on top. To enable the 5V Pin, either one of these need to be connected If you power the board from the VIN pin, you won't get any regulated 5V and therefore even if you do the solder bridge, nothing will come out of that 5V pin. The 3.3V, on the other hand, is always available and supports enough current to drive your sensors. Please make your designs so that sensors and actuators are driven with 3.3V and work with 3.3V digital IO levels. 5V is now an option for many modules and 3.3V is becoming the standard voltage for electronic ICs.



Soldering the VUSB pins.

IMU



The LSM9DS1 sensor

BMI270 and BMM150

The Arduino Nano 33 BLE Sense Rev2 Inertial Measurement Unit system is made up of two separate IMUs, a 6-axis BMI270 and a 3-axis BMM150, effectively giving you a 9-axis IMU system. This allows you to detect orientation, motion, or vibrations in your project.

BMI270 and BMM150 Library

To access the data from the IMU system, we need to install the **BMI270_BMM150** library, which comes with examples that can be used directly with the Nano 33 BLE Sense Rev2.

It can be installed directly from the library manager through the IDE of your choice. To use it, we need to include it at the top of the sketch:

```
#include "Arduino_BMI270_
```

And to initialize the library, we can use the following command inside `void setup()` .

```
if (!IMU.begin()) {
    Serial.println("Faile
    while (1);
  }
```

Accelerometer

The accelerometer data can be accessed through the following commands:

```
float x, y, z;

  if (IMU.accelerationAva
    IMU.readAcceleration(
  }
```

Gyroscope

The gyroscope data can be accessed through the following commands:

```
float x, y, z;

  if (IMU.gyroscopeAvaila
    IMU.readGyroscope(x,
  }
```

The magnetometer data can be accessed through the following commands:

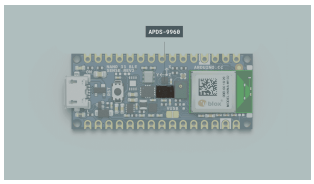
```
float x, y, z;

  IMU.readMagneticField(x
```

Tutorials

If you want to learn more on how to use the IMU, please check out the tutorial below:

- Accessing IMU gyroscope data with Nano 33 BLE Sense Rev2
- Accessing IMU accelerometer data with Nano 33 BLE Sense Rev2
- Accessing IMU magnetometer data with Nano 33 BLE Sense Rev2

Proximity and Gesture Detection



The APDS-9960 proximity and gesture sensor

APDS9960

The **APDS9960** chip allows for

ambient light as well as for detecting RGB colors and gestures.

APDS9960 Library

To access the data from the APDS9960 module, we need to install the **APDS9960** library, which comes with examples that can be used directly with the Nano 33 BLE Sense Rev2.

It can be installed directly from the library manager through the IDE of your choice. To use it, we need to include it at the top of the sketch:

```
#include <Arduino_APDS996
```

And to initialize the library, we can use the following command inside `void setup()` .

```
if (!APDS.begin()) {
  Serial.println("Error i
}
```

Then we check if there is data available from the proximity sensor. If there is we can print the value in the serial monitor. The value can range between 0-255, where 0 is close and 255 is far away. If it prints the value -1, it indicates an error.

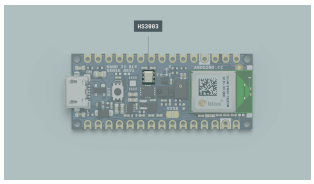
```
if (APDS.proximityAvailab
  Serial.println(APDS.rea
}
```

Tutorials

If you want to learn more on how to use the proximity sensor, please check out the tutorial below:

- Proximity Detection with the Nano 33 BLE Sense
- Gesture Recognition with the Nano 33 BLE Sense

Temperature and Humidity Sensor



The HS3003 temperature and humidity sensor

HS3003

The **HS3003** capacitive digital sensor measures relative humidity and temperature. It has a temperature accuracy of ± 0.5 °C (between 15-40 °C) and is thereby perfectly suited to detect ambient temperature.

HS3003 Library

To access the data from the HS3003 module, we need to install the Arduino_HS300x library, which comes with examples that can be used directly with the Nano 33 BLE Sense Rev2.

It can be installed directly from the

your choice. To use it, we need to include it at the top of the sketch:

```
#include <Arduino_HS300x.
```

And to initialize the library, we can use the following command inside `void setup()` .

```
if (!HS300x.begin()) {
  Serial.println("Failed
}
```

Then we can print our values in the serial monitor to check the temperature and humidity values.

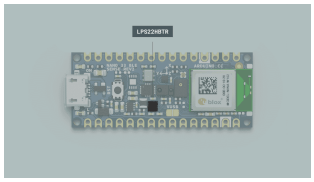
```
Serial.println(HS300x.rea
Serial.println(HS300x.rea
```

Tutorial

If you want to learn more on how to use the temperature and humidity sensor, please check out the tutorial below:

- Reading Temperature & Humidity on Nano 33 BLE Sense Rev2

Pressure Sensor



The LPS22HB pressure sensor

LPS22HB

The **LPS22HB** picks up on barometric pressure and allows for a 24-bit pressure data output between 260 to 1260 hPa. This data can also be processed to calculate the height above sea level of the current location.

LPS22HB Library

To access the data from the LPS22HB module, we need to install the **LPS22HB** library, which comes with examples that can be used directly with the Nano 33 BLE Sense Rev2.

It can be installed directly from the library manager through the IDE of your choice. To use it, we need to include it at the top of the sketch:

```
#include <Arduino_LPS22HB
```

And to initialize the library, we can use the following command inside `void setup()` .

```
if (!BARO.begin()) {
  Serial.println("Failed
}
```

Then we can read the values from the sensor using the code below.

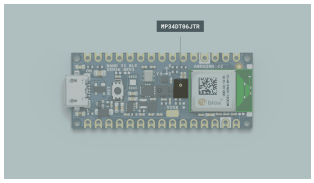
```
BARO.readPressure();
```

Tutorial

If you want to learn more on how to use the temperature and humidity sensor, please check out the tutorial below:

- Access Barometric Pressure Sensor Data on Nano 33 BLE Sense Rev2

Microphone



The MP34DT05 microphone

MP34DT06JTR

The **MP34DT06JTR** is a compact, low-power omnidirectional digital MEMS microphone with an IC interface. It has a 64 dB signal-to-noise ratio, is capable of sensing acoustic waves and can operate in temperatures of

PDM Library

To access the data from the MP34DT06JTR, we need to use the PDM library that is included in the **Arduino Mbed OS Nano Board package**. If the Board Package is installed, you will find an example that works by browsing **File > Examples > PDM > PDMSerialPlotter**.

Please note: The sampling frequency in the PDMSerialPlotter example is set to 16000 Hz. If the microphone appears to not be working (monitor is printing a value of -128), try to change this rate to 20000 Hz. You can change this at the top of the PDMSerialPlotter example sketch.

```
static const int frequenc
```

Tutorial

If you want to learn more on how to use the Microphone, please check out the tutorial below:

Controlling the On-Board RGB LED with Microphone

RGB

To turn ON the pixels, write a `HIGH` state to the LED:

```
digitalWrite(LEDR, HIGH);
digitalWrite(LEDG, HIGH);
digitalWrite(LEDB, HIGH);
```

To turn OFF the pixels, write a `LOW` state to the LED:

```
digitalWrite(LEDR, LOW);
digitalWrite(LEDG, LOW);
digitalWrite(LEDB, LOW);
```

We can also choose a value between 255 - 0 to write to the LED:

```
analogWrite(LEDR, 72);  /
analogWrite(LEDG, 122); /
analogWrite(LEDB, 234); /
```

Communication

Like other Arduino® products, the Nano 33 BLE Sense Rev2 features dedicated pins for different protocols.

SPI

The pins used for SPI (Serial Peripheral Interface) on the Nano 33 BLE Sense Rev2 are the following:

(CIPO) - D12

(COPI) - D11

(SCK) - D13

(CS/SS) - Any GPIO

The signal names MOSI, MISO and SS has been replaced by COPI (Controller Out, Peripheral In), CIPO (Controller In, Peripheral Out) and CS (Chip Select).

To use SPI, we first need to include the **SPI** library.

```
#include <SPI.h>
```

Inside `void setup()` we need to initialize the library.

```
SPI.begin();
```

And to write to the device:

```
digitalWrite(chipSelectPi

  SPI.transfer(address);
  SPI.transfer(value); //

  digitalWrite(chipSelect
```

I2C

The pins used for I2C (Inter-Integrated Circuit) on the Nano 33 BLE Sense Rev2 are the following:

(SDA) - A4

(SCL) - A5

To use I2C, we can use the **Wire** library, which we need to include at the top of our sketch

```
#include <Wire.h>
```

Inside `void setup()` we need to initialize the library.

```
Wire.begin();
```

And to write something to a device connected via I2C, we can use the following commands:

```
Wire.beginTransmission(1)
  Wire.write(byte(0x00));
  Wire.write(val); //send
  Wire.endTransmission();
```

UART

The pins used for UART (Universal asynchronous receiver-transmitter) are the following:

(Rx) - D0

(Tx) - D1

To send and receive data through UART, we will first need to set the baud rate inside `void setup()` .

```
Serial1.begin(9600);
```

To read incoming data, we can use a while loop() to read each individual character and add it to a string.

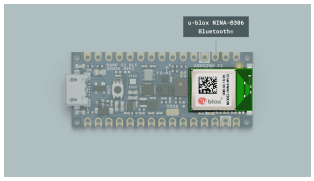
```
while(Serial1.available()
    delay(2);
    char c = Serial1.read
    incoming += c;
  }
```

And to write something, we can use the following command:

```
Serial1.write("Hello worl
```

Connectivity

The Nano 33 BLE Sense Rev2 supports Bluetooth® through the **u-blox NINA-B306** module. To use this module, we can use the **ArduinoBLE** library.



Bluetooth® module.

Bluetooth®

To enable Bluetooth® on the Nano 33 BLE Sense Rev2, we can use the **ArduinoBLE** library, and include it at the top of our sketch:

```
#include <ArduinoBLE.h>
```

```
BLEService ledService("18
BLEByteCharacteristic swi
```

Set advertised name and service:

```
BLE.setLocalName("Nano 33
  BLE.setAdvertisedServic
```

Start advertising:

```
BLE.advertise();
```

Listen for Bluetooth® Low Energy peripherals to connect:

```
BLEDevice central = BLE.c
```

Tutorials

- Controlling Nano 33 BLE Sense Rev2 RGB LED via Bluetooth®

USB Keyboard

To use the board as a keyboard, you can refer to the **USBHID** library that can be found inside the Board Package.

You first need to include the libraries and create an object:

```
#include "PluggableUSBHID
#include "USBKeyboard.h"

USBKeyboard Keyboard;
```

Then use the following command to write something:

```
Keyboard.printf("This is
```

Suggest changes

The content on docs.arduino.cc is facilitated through a public GitHub repository. If you see anything wrong, you can edit this page here.

Need support?

Help Center
Ask the Arduino Forum
Discover Arduino Discord

License

The Arduino documentation is licensed under the Creative Commons Attribution-Share Alike 4.0 license.

Was this article helpful?

© 2024 Arduino